

Mini Banking & Payments System

Python OOP + Exception Handling — Coding Test

FIL Fresher Training · 90 minutes · Standard library only

Focus: Correctness • Clarity • Idiomatic Python • Best Practices

1. What you are building

A small banking core: money that cannot lose paise to floating-point error, accounts that cannot be corrupted from outside, transfers that either happen completely or not at all, and month-end processing that can be extended without editing existing code.

The domain is small on purpose. The marks are in **how** you build it.

You are given a starter repository with every module, class, method and docstring already in place. Each body you must write says `raise NotImplementedError("TODO")`. **Do not change any signature, class name or module name** — the test suite imports them by name.

```
mini_banking_exercise/
├── EXERCISE.md          <- this document
├── README.md           <- how to run things
├── requirements.txt
├── pytest.ini
├── demo.py             <- end-to-end smoke script (not graded)
├── minibank/
│   ├── __init__.py     GIVEN   public API
│   ├── exceptions.py   GIVEN   the full exception hierarchy
│   ├── money.py        TASK 1  value object, operator overloading
│   ├── transaction.py  TASK 2  immutable ledger record (enum GIVEN)
│   ├── mixins.py       TASK 3  multiple inheritance
│   ├── accounts.py     TASK 4  abstract base + two concrete types
│   ├── strategies.py   TASK 5  duck-typed month-end strategies
│   └── bank.py         TASK 6  factory + orchestration
└── tests/             GIVEN   171 tests. Do not edit.
```

Definition of done: `pytest` reports 171 passed.

On a fresh clone 14 already pass (they exercise the given code). The other 157 are yours.

2. The rules

- 1. **Standard library only.** `decimal`, `abc`, `enum`, `dataclasses`, `functools`, `itertools`, `json`, `re`, `datetime`. Nothing else is needed.
- 2. **Never `float` for money.** Not once, not “just for the interest rate”.
- 3. **No bare `except:`** and no `except Exception:`. Catch the narrowest exception that can actually occur.
- 4. **Chain wrapped exceptions** with `raise NewError(...) from exc`.
- 5. **No public mutable state.** No public setter for a balance, ever.
- 6. **Do not edit `tests/`.** Private helper methods inside the given modules are welcome; new public methods are not needed.
- 7. **Docstrings on every public method.** One line is enough where the docstring already given is accurate — keep it.

A solution that passes every test while breaking rules 2, 3 or 5 loses more marks than a solution that misses a few tests cleanly.

3. Task 1 — Money (20 min)

A value object wrapping `Decimal` + a 3-letter currency code.

Why

`0.1 + 0.2 != 0.3` in binary floating point. A bank that adds interest in `float` will not balance. `Decimal` is exact for decimal fractions, which is what money is.

Invariants the tests check

#	Invariant
I1	<code>amount</code> is a <code>Decimal</code> quantised to exactly 2 places, <code>ROUND_HALF_UP</code> . <code>Money("100.005").amount == Decimal("100.01")</code>
I2	<code>currency</code> is 3 uppercase letters. <code>"inr" → "INR"</code> ; <code>"IN"</code> , <code>"RUPEE"</code> , <code>"1NR"</code> , <code>""</code> → <code>InvalidAmountError</code>
I3	Instances are immutable — assigning to <code>m.amount</code> raises
I4	Instances are hashable, and equal instances hash equally
I5	Negative and zero amounts are legal (a checking account can be overdrawn). Only <i>unparseable input</i> is rejected
I6	<code>+</code> , <code>-</code> and ordering across currencies raise <code>CurrencyMismatchError</code>

`Money` does **not** enforce business rules. “You cannot deposit ₹0” is an *account* rule, not a *Money* rule. Keeping validation at the right layer is part of the grade.

Constructor behaviour

Accepts `int`, `str`, `Decimal`. `float` is accepted only by going through `str(value)` first — `Decimal(0.1)` is `0.1000000000000000055511151231257827`, which defeats the point. Anything unparseable raises `InvalidAmountError` chained from the `decimal.InvalidOperation`.

Operators

Expression	Result
<code>Money("100.50") + Money("9.50")</code>	<code>Money('110.00', 'INR')</code>
<code>Money("100") - Money("150")</code>	<code>Money('-50.00', 'INR')</code>
<code>Money("100") * 3, 3 * Money("100")</code>	<code>Money('300.00', 'INR')</code>
<code>Money("100") * Decimal("0.04")</code>	<code>Money('4.00', 'INR')</code>
<code>Money("100") * Money("2")</code>	<code>TypeError</code>
<code>Money("100") + 5</code>	<code>TypeError</code>
<code>-Money("10"), abs(Money("-10"))</code>	<code>Money('-10.00'), Money('10.00')</code>
<code>Money("0") == 0</code>	<code>False</code> — never an exception
<code>Money("1", "INR") < Money("1", "USD")</code>	<code>CurrencyMismatchError</code>
<code>repr(Money("110.5", "usd"))</code>	<code>"Money('110.50', 'USD')"</code>
<code>str(Money("1234.5"))</code>	<code>"INR 1234.50"</code>
<code>bool(Money("0"))</code>	<code>False</code>

Three things that separate a good answer from a passing one

- **Return `NotImplemented`, do not raise**, when the other operand is the wrong type. That is the protocol: Python then tries the reflected operation and raises `TypeError` itself. Raising `TypeError` directly breaks interoperability with types you have never heard of.
- **Implement `__lt__` only** and decorate the class with `functools.total_ordering` to get `<=`, `>`, `>=`. Writing all four by hand is four chances to get a sign wrong.
- **Use `@dataclass(frozen=True)`** and normalise in `__post_init__` via `object.__setattr__`. That is the idiomatic way to have an immutable type that still cleans its inputs.

4. Task 2 — Transaction (10 min)

An immutable, hashable record of something that happened.

`TransactionType` is **given**: `DEPOSIT`, `WITHDRAWAL`, `TRANSFER_IN`, `TRANSFER_OUT`, `INTEREST`, `FEE`, with an `is_credit` property. Use it.

Fields

Field	Notes
<code>txn_id</code>	Unique; generated by the account, not the caller
<code>type</code>	A <code>TransactionType</code> member. Anything else → <code>InvalidAmountError</code>
<code>amount</code>	Always a positive <code>Money</code> . Direction lives in <code>type</code> , not in the sign
<code>balance_after</code>	Balance once this entry was applied — makes the ledger auditable without replaying it
<code>description</code>	Free text, may be empty
<code>timestamp</code>	Defaults to timezone- aware UTC. A naive datetime in a financial record is a bug

Required behaviour

- `signed_amount` → `+amount` for credits, `-amount` for debits.
- `__hash__` and `__eq__` both key on `txn_id` alone. Two objects with the same id are the same record, whatever else differs — this is what lets you drop a replayed batch into a `set` and de-duplicate it.
- `__repr__` contains the id, the type name and the amount.
- `to_dict()` returns **exactly** these keys:

```
{
  "txn_id": str,
  "type": str,
  "amount": str,
  "currency": str,
  "balance_after": str,
  "description": str,
  "timestamp": str
}
```

`amount` and `balance_after` are plain decimal strings (`"100.00"`, no currency prefix); `timestamp` is `datetime.isoformat()`.

5. Task 3 — Mixins (10 min)

Two mixins, each adding exactly one capability.

JSONSerializableMixin

Method	Contract
<code>to_dict()</code>	Hook for the host class. The mixin's own version raises <code>NotImplementedError</code> naming the class: <code>"SavingsAccount must implement to_dict()"</code>
<code>to_json(*, indent=None)</code>	<code>json.dumps</code> of <code>to_dict()</code> , with <code>default=str</code> so <code>Decimal</code> and <code>datetime</code> do not blow up
<code>from_json(payload)</code> (classmethod)	Returns a dict. Invalid JSON → <code>BankError</code> chained from the <code>json.JSONDecodeError</code>

AuditableMixin

An append-only trail of human-readable event strings. Deliberately separate from the transaction ledger: the ledger records **money**, the trail records **events** (“opened”, “closed”, “withdrawal rejected: insufficient funds”).

Member	Contract
<code>__init__(*args, **kwargs)</code>	Create the private list, then <code>super().__init__(*args, **kwargs)</code>
<code>audit(message)</code>	Append an ISO-timestamped entry
<code>audit_trail</code>	Read-only tuple snapshot

The one thing that is actually being tested

`Account` is declared as:

```
class Account(JSONSerializableMixin, AuditableMixin, ABC):
```

so `Account.__mro__` is `Account → JSONSerializableMixin → AuditableMixin → ABC → object`.

If `AuditableMixin.__init__` forgets `super().__init__(...)`, the chain stops there and everything after it is silently skipped. That is *the* classic multiple-inheritance bug, it is why cooperative `super()` exists, and you should be able to explain it at the review. Print `Account.__mro__` and look.

6. Task 4 — The account hierarchy (25 min)

Account (ABC, + JSONSerializableMixin, AuditableMixin)

├ InterestBearingAccount (ABC)

├ SavingsAccount

└ CheckingAccount

adds annual_interest_rate + accrual

minimum balance, earns interest

overdraft allowed, monthly fee

`InterestBearingAccount` exists to prove you can put an abstract class *between* levels rather than pushing interest onto every account. A checking account must **not** have an `accrue_monthly_interest` method at all.

Encapsulation (graded hard)

`_balance` and `_ledger` are private. There is **no** public setter. The only things that change the balance are `deposit`, `withdraw`, `transfer_out`, `transfer_in` and the protected `_apply` used for interest and fees.

- `account.balance` → read-only property. `account.balance = ...` must raise.
- `account.ledger` → a **tuple**, so a caller cannot append to the real list.
- No public attributes in `vars(account)` at all. A test asserts this.

The abstract contract

Three members every concrete subclass must supply:

Member	Savings	Checking
<code>account_type</code>	<code>"SAVINGS"</code>	<code>"CHECKING"</code>
<code>withdrawable_balance</code>	<code>balance - minimum_balance</code> , floored at zero	<code>balance + overdraft_limit</code>
<code>_insufficient_funds_error(requested)</code>	<code>InsufficientFundsError</code>	<code>OverdraftLimitExceededError</code> (carries <code>limit</code>)

`withdraw` is written once, in the base class, in terms of `withdrawable_balance`. That is the polymorphism this task is testing. A test asserts `withdraw` does not appear in either subclass's `_dict__`. Same idea for `deposit`, `close`, `_len__`, `_repr__`, `to_dict`.

Money movement

`_validate(amount)` is the shared guard. It raises **in this order**:

1. `AccountClosedError` — the account is closed
2. `InvalidAmountError` — not a `Money` instance
3. `CurrencyMismatchError` — wrong currency
4. `InvalidAmountError` — not strictly positive

`_apply(txn_type, amount, description)` is the single place `_balance` is written. Credits add, debits subtract; it appends the `Transaction` and returns it. Everything else goes through it.

Method	Ledger entry	Extra rule
<code>deposit</code>	<code>DEPOSIT</code>	—
<code>withdraw</code>	<code>WITHDRAWAL</code>	refuse if <code>amount > withdrawable_balance</code>
<code>transfer_out</code>	<code>TRANSFER_OUT</code>	same limit check as <code>withdraw</code>
<code>transfer_in</code>	<code>TRANSFER_IN</code>	same guard as <code>deposit</code>

`withdraw` and `transfer_out` share the limit check — extract one private helper. Copy-pasting the body costs marks.

A refused withdrawal changes nothing. Not the balance, not the ledger. It *does* write an audit entry containing the word “reject”, then raises `self._insufficient_funds_error(amount)`.

Defaults

	Savings	Checking
minimum balance	<code>Money("1000")</code>	—
overdraft limit	—	<code>Money("5000")</code>
annual interest	<code>Decimal("0.04")</code>	none
monthly fee	<code>Money("0")</code>	<code>Money("150")</code>

All overridable per account via constructor keywords. Negative values → `InvalidAmountError`.

Interest

```
monthly_interest() = balance * annual_interest_rate / 12
```

Return `Money.zero(currency)` when the balance is not positive — **never pay interest on an overdrawn account**. Rounding is whatever `Money` already does.

`accrue_monthly_interest()` credits it and returns the `INTEREST` transaction, or `None` when the interest is zero or the account is closed. No zero-value ledger noise.

Worked example: balance ₹12,000 at 12% p.a. → $12000 \times 0.12 / 12 = \text{₹120.00}$, new balance ₹12,120.00.

Lifecycle

`close()` is idempotent, writes an audit entry, and may leave a non-zero balance (the Bank decides policy). After closing, `deposit` and `withdraw` raise `AccountClosedError`.

Dunders

`__len__` (transaction count), `__iter__` (over the ledger), `__repr__` (`SavingsAccount(id='SB0001', owner='Asha', balance=INR 5000.00)` — use `type(self).__name__` so subclasses need no override), `__eq__` / `__hash__` (identity is the `account_id`, so a savings and a checking account with the same id are “equal” — ids are unique in practice).

to_dict

Base keys:

```
{"account_id", "account_type", "owner", "currency",
 "balance", "is_closed", "transactions"}
```

`balance` is a decimal string; `transactions` is a list of transaction dicts. Savings adds `minimum_balance` and `interest_rate`; Checking adds `overdraft_limit` and `monthly_fee` — all strings. **Call `super().to_dict()` and add to it.** Rebuilding the common keys costs marks.

7. Task 5 — Month-end strategies (10 min)

A strategy is **any** object shaped like this:

```
class SomeStrategy:
    name: str
    def apply(self, account) -> list[Transaction]: ...
```

There is deliberately **no abstract base class**. That is the point: Python does not need one. `Bank.monthly_process` validates the *shape* with `getattr/callable`, not the *type*.

Every `apply` must return a list (possibly empty) of the transactions it created, skip closed accounts, and never raise for a normal no-op.

Strategy	name	Behaviour
InterestStrategy	"interest"	Accrue interest if the account is an <code>InterestBearingAccount</code> and open. Anything else → <code>[]</code>
MaintenanceFeeStrategy	"maintenance-fee"	Charge <code>account.monthly_fee()</code> . Fee $\leq 0 \rightarrow []$. Charge it through <code>account.withdraw</code> so the account's own limits apply. If refused, catch <code>InsufficientFundsError</code> <i>specifically</i> , audit it, return <code>[]</code>
CompositeStrategy	children joined with <code>"+"</code>	Run children in order, concatenate results. Reject an object with no <code>apply</code> at construction time with <code>TypeError</code>

`InterestStrategy` is the one place an explicit `isinstance` check is the right call, because the capability *is* defined by the abstract class. Everywhere else, ask for behaviour, not for a type.

`CompositeStrategy` is composition beating inheritance: no strategy subclasses another, they are held in a list, and they nest.

Why route the fee through `withdraw` rather than `_apply`? Because a fee must not be able to break a rule the account guarantees. A ₹150 fee on a checking account with ₹100 and a ₹40 overdraft limit must be **refused**, not forced through. A test checks exactly that.

8. Task 6 — The Bank orchestrator (15 min)

The Bank never touches `account._balance`. It only calls the public account API. That is what makes the encapsulation real rather than decorative.

Factory

`ACCOUNT_TYPES` is a class-level registry: `{"SAVINGS": SavingsAccount, "CHECKING": CheckingAccount}`.

```
bank.open_account("CHECKING", "Ravi", Money("2000"),
                  overdraft_limit=Money("10000"))
```

- `kind` matched case-insensitively; unknown → `UnknownAccountTypeError`.
- `**kwargs` forwarded to the concrete class — so the Bank does not need to know what a checking account is.
- Ids generated per kind: `SB0001`, `SB0002`, `CH0001`, ...
- Opening balance in a foreign currency → `CurrencyMismatchError`.

`register_account_type(kind, cls)` (a `@classmethod`) adds a type. **Adding a new account type must not require editing `open_account`** — that is the open/closed principle, and a test registers a new subclass to prove it. A non-`Account` class → `TypeError`.

`validate_account_id(id)` is a `@staticmethod` (it needs neither instance nor class state — that *is* the lesson). Valid shape: two letters then four digits. Returns it upper-cased; anything else → `ValueError`.

Transfers — atomicity

```
debit, credit = bank.transfer(from_id, to_id, amount, description)
```

Situation	Result
<code>from_id == to_id</code>	<code>TransferError</code> , nothing recorded
either account closed	<code>AccountClosedError</code> , nothing recorded
currencies differ	<code>CurrencyMismatchError</code> , nothing recorded
source cannot afford it	the <code>InsufficientFundsError</code> from the debit propagates unchanged, nothing recorded anywhere
debit succeeds, credit then fails	put the money back with a compensating <code>transfer_in</code> on the source, then <code>raise TransferError(...)</code> from <code>exc</code>

A passing transfer conserves `total_assets()` exactly. The test for the last row monkeypatches the destination's `transfer_in` to explode and then asserts the source balance is untouched — this is the one place in the exercise where you must think like someone who has seen a half-completed transfer in production.

Reporting

`total_assets()` sums the balances of all **open** accounts in the bank's base currency; accounts in another currency are skipped (no FX here). Overdrawn accounts count as negative. Empty bank → `Money.zero(currency)`. Build the sum out of `Money.__add__` — do not reach for `Decimal`.

`monthly_process(strategy)`

- Validate the shape: callable `apply` plus a `name`, else `TypeError` naming what was missing. Do **not** require a base class.

2. Return `{account_id: [transactions created]}`, including empty lists.
3. A `BankError` from one account must not abort the run: record it in that account’s audit trail and carry on. One account’s bad month is not the bank’s bad month.

Dunders

`__len__`, `__contains__` (accepts an id string or an `Account`), `__iter__`, `__getitem__` (same as `get_account`), `__repr__` (`Bank(name='FIL Bank', accounts=3, assets=INR 12500.00)`), and `to_dict` with keys `{"name", "currency", "total_assets", "accounts"}`.

9. The exception hierarchy (given — read it)

```
BankError
├── CurrencyMismatchError           .left .right
├── InvalidAmountError (+ValueError) .value .reason
├── InsufficientFundsError          .requested .available
│   └── OverdraftLimitExceededError + .limit
├── AccountError
│   ├── AccountNotFoundError (+KeyError) .account_id
│   ├── DuplicateAccountError          .account_id
│   └── AccountClosedError             .account_id
├── TransferError
└── UnknownAccountTypeError (+ValueError) .kind
```

Three design decisions worth understanding, because you will be asked:

- **Everything derives from `BankError`**, so a caller can write `except BankError` and be sure. A library that raises bare `ValueError` from six places is untestable.
- **`OverdraftLimitExceededError` subclasses `InsufficientFundsError`**. Callers that only care about “not enough money” keep working; callers that care about the overdraft specifically can catch the narrower type. Design your hierarchy for the `except` clauses people will actually write.
- **Multiple inheritance from builtins** (`InvalidAmountError(BankError, ValueError)`) lets generic numeric code keep working while domain code stays specific. Use it sparingly and on purpose.

Exceptions carry **data**, not just a message: `exc.available` is what lets a caller tell the customer how much they *can* withdraw.

10. Marking scheme (100)

Area	Marks	What earns them
Tests passing	40	Proportional. 171/171 = 40
Encapsulation	10	Private state, read-only properties, tuple snapshots, no leaked internals
Abstraction & polymorphism	10	<code>withdraw</code> written once; no <code>isinstance</code> chains where polymorphism belongs; the abstract mid-layer used properly
Operator overloading	10	<code>NotImplemented</code> not <code>TypeError</code> ; <code>total_ordering</code> ; consistent <code>__eq__</code> / <code>__hash__</code>
Exception handling	10	Narrow <code>except</code> , <code>raise ... from</code> , no bare <code>except</code> , exceptions carry data, atomic transfer
Idiomatic Python	10	<code>Decimal</code> throughout, dataclasses, enums, <code>@property</code> / <code>@classmethod</code> / <code>@staticmethod</code> used for the right reasons, no duplicated logic
Clarity	10	Docstrings, readable names, short methods, <code>NOTES.md</code>

Automatic deductions: any `float` in a money path (−10); any bare `except:` or `except Exception:` (−10); a public balance setter (−10); editing `tests/` (−20).

11. Timeboxing

Minutes	Do this
0-5	Read this document. Run <code>pytest</code> and watch it fail. Skim <code>exceptions.py</code>
5-25	Task 1 — <code>pytest tests/test_money.py</code> green
25-35	Task 2
35-45	Task 3
45-70	Task 4 (the big one)
70-80	Task 5
80-90	Task 6, then <code>python demo.py</code> , then <code>NOTES.md</code>

If you fall behind, **keep what you have green**. A clean solution to five tasks beats six broken ones. Write `NOTES.md` even if you are out of time — naming what you would do next is worth more than a half-finished module.

12. Submitting

The repository, plus a short `NOTES.md`:

1. One design decision you made and the alternative you rejected.
2. Where you used composition instead of inheritance, and why.
3. Anything unfinished, and what you would do next.

Appendix A — Commands

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

pytest                                # all 171
pytest tests/test_money.py           # one task
pytest -x -q                          # stop at first failure
pytest -k overdraft                  # one behaviour
pytest --lf                          # only what failed last time

python demo.py                        # end-to-end smoke test
```

Appendix B — Worked end-to-end example

```
from decimal import Decimal
from minibank import Bank, Money, InterestStrategy, MaintenanceFeeStrategy, CompositeStrategy

bank = Bank("FIL Bank")
savings = bank.open_account("SAVINGS", "Asha", Money("50000"), interest_rate=Decimal("0.06"))
checking = bank.open_account("CHECKING", "Ravi", Money("8000"))

bank.transfer(savings.account_id, checking.account_id, Money("12000"), "rent")
# savings INR 38000.00    checking INR 20000.00

savings.withdraw(Money("999999"))
# InsufficientFundsError: requested INR 999999.00, only INR 37000.00 available
#   (50000 - 12000 - 1000 minimum balance)

checking.withdraw(Money("999999"))
# OverdraftLimitExceededError: requested INR 999999.00, only INR 25000.00 available
#   (20000 balance + 5000 overdraft limit)

bank.monthly_process(CompositeStrategy(InterestStrategy(), MaintenanceFeeStrategy()))
# {'SB0001': [Transaction(T..., INTEREST, INR 190.00)],    38000 × 0.06/12
#  'CH0001': [Transaction(T..., WITHDRAWAL, INR 150.00)]}  the maintenance fee

bank.total_assets()      # Money('58040.00', 'INR')
print(savings.statement())
```

Appendix C — Concept checklist

Tick these off before you submit; each one is somewhere in the tests.

- **Encapsulation** — private attributes · read-only properties · defensive copies (`tuple`) · no public setters
- **Inheritance** — abstract base with `ABC` / `@abstractmethod` · an abstract mid-layer · `super()` used for real, including in `to_dict`
- **Polymorphism** — one `withdraw` for every account type · duck-typed strategies · an `_eq` that refuses to guess
- **Abstraction** — behaviour named at the level it belongs to (`withdrawable_balance`, `_insufficient_funds_error`)
- **Operator overloading** — `_add` `_sub` `_mul` `_rmul` `_neg` `_abs` `_eq` `_lt` `_hash` `_repr` `_str` `_bool`

`__len__` `__iter__` `__contains__` `__getitem__`

- **Multiple inheritance** — two mixins · a readable MRO · cooperative `super().__init__`
- **Decorators** — `@property` · `@classmethod` (factory registry) · `@staticmethod` (pure validator) · `@abstractmethod` · `@dataclass(frozen=True)` · `@functools.total_ordering`
- **Exceptions** — a rooted hierarchy · data on the exception · narrow `except` · `raise ... from` · a compensating rollback